GÖRÜNTÜ İŞLEME - (7.Hafta)

KENAR BULMA ALGORİTMALARI

Bu konuda bir çok algoritma olmasına rağmen en yaygın kullanılan ve etkili olan Sobel algoritması burada anlatılacaktır.

Sobel KenarBulma Algoritması

Görüntüyü siyah beyaza çevirdikten sonra eğer kenar bulma algoritmalarını kullanmak isterseniz bir kaç seçenekten en popüleri sobel kenar bulma filtresidir. Aşağıdaki çekirdek matrisler (konvolüsyon matrisleri) dikey, yatay ve köşegen şeklindeki kenarları bulmak için kullanılır. Sobel operatörü bir resmin kenarlarına karşılık gelen alansal yüksek frekans bölgelerini (keskin kenarları) ortaya çıkarır. Teorik olarak, operatör aşağıda gösterildiği gibi 3×3 konvolüsyon matrisinden oluşur.

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

Şekil. Sobel konvolüsyon matrisi

Bu matrisler, yatay ve dikey olarak görünen kenarlar için ayrı ayrı olacak şekilde düzenlenmiştir. Matrisler giriş görüntüsüne birbirinden bağımsız uygulanabilir. Böylece her bir yön için pikselin değeri ayrı ayrı ölçülmüş olur. Daha sonra çapraz duran kenarların mutlak değerini bulmak ve yönünü (açısını) bulmak için bu değerler aşağıdaki formüllerle birleştirilebilir. Piksel mutlak değeri şu şekilde hesaplanabilir.

$$|G| = \sqrt{Gx^2 + Gy^2}$$

Piksel değerini daha hızlı hesaplama için şu formülde kullanılabilir.

$$|G| = |Gx| + |Gy|$$

Ortaaya çıkan kenarın yön açısı (piksel ızgarasına göre) x ve y yönlerindeki değerlerine bakarak şu şekilde bulunur.

$$\theta = \arctan\left(\frac{|Gx|}{|Gy|}\right)$$

(Gx dikey çizgiyi bulur, Gy yatay çizgiyi bulur. Tan karşı kenarın komşu kenara oranı olduğu için formül bu şekilde yazıldı)

Bu durumda 0 derece yatay duran çizgileri, 90 derece ise dikey duran çizgileri gösterecektir. Eğik duran çizgilerde diğer açıları oluşturacaktır. Burada sıfır derece çizgide alt kısım siyahtan, üst kısım beyaza doğru geçişi gösterir. 0 derecenin bir benzeri olan yine yatay duran çizgi 180 derecede ise üst kısım siyah bölgeden alt kısım beyaz bölgeye doğru bir geçişi gösterecektir. Açılar saatin tersi yönüne göre ölçülür.

Burada Gx konvolüsyon matrisi tek başına kullanılırsa yatayda renk değişimini bulacağından, ortaya çıkan çizgileri dikey olarak görürüz. Benzer şekilde Gy konvolüsyon matrisi de tek başına kullanılırsa aşağıdan yukarıya doğru renk geçişini (siyahtan-beyaza) göstereceğinden ortaya çıkan çizgiler yatay olacaktır. Resim üzerindeki çizgileri yatay yada dikey görmek yerine kendi doğal duruşlarını görmek istersek, her iki konvolüsyon matrisini yukarıda formülü verilen hesaplama ile toplayabiliriz. Benzer şekilde yukarıda verilen karaköklü formülü de kullanabiliriz. Eğer resim üzerinde belli açılardaki çizgileri ortaya çıkarmak istersek verilen açı formülü ile hesaplama yaparak bu çizgileri de belirleyebiliriz.

İki matrisi aynı anda kullanmak için ve matris üzerindeki noktaları aşağıdaki şekilde temsil etmek için hesaplama yaparak deneyelim.

P_1	P_2	P_3
P_4	P_5	P_6
P_7	P_8	P_9

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

Buradaki her bir çekirdek matrisin hesabı şu şekilde hesaplanır.

$$|Gx| = |-P_1 + P_3 - 2P_4 + 2P_6 - P_7 + P_9|$$

$$|Gy| = |P_1 + 2P_2 + P_3 - P_7 - 2P_8 - P_9|$$

Bu çekirdek matris kullanılarak pikselin sobel değeri yaklaşık formül kullanılarak şu şekilde hesaplanır:

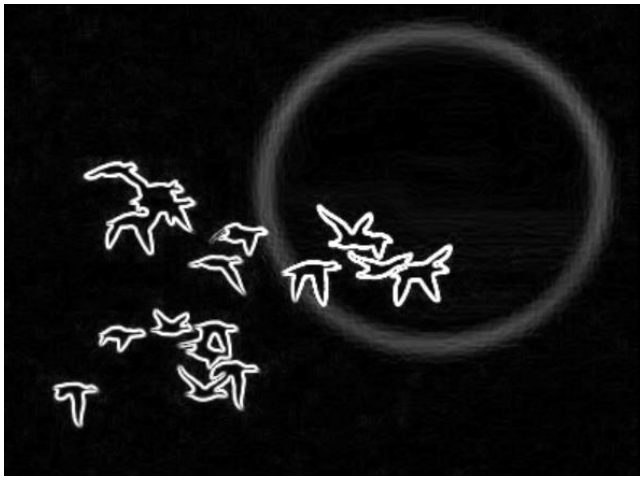
$$|G| = |Gx| + |Gy|$$

Diğer formül kullanıldığında ise şu şekilde olacaktır.

$$|G| = \sqrt{Gx^2 + Gy^2}$$

Burada renkli resimde renk değerleri 3 kanal olduğunda tek kanal üzerinden işlem yapmak için Gri renk değerleri üzerinden işlem yapmak gerekir. Aşağıdaki programda Gri renk için 3 kanalın ortalması alınmıştır. Gerekirse Gri renk formülleri de kullanılabilir (daha önceki konularda geçti).

Programlama (Sobel Filtresi)



1. Kodlama

```
private void mnuSobel_Click(object sender, EventArgs e)
{
    Bitmap GirisResmi, CikisResmiXY, CikisResmiX, CikisResmiY;
    GirisResmi = new Bitmap(pictureBox1.Image);

    int ResimGenisligi = GirisResmi.Width;
    int ResimYuksekligi = GirisResmi.Height;

    CikisResmiX = new Bitmap(ResimGenisligi, ResimYuksekligi);
    CikisResmiY = new Bitmap(ResimGenisligi, ResimYuksekligi);
    CikisResmiXY = new Bitmap(ResimGenisligi, ResimYuksekligi);
```

```
    int SablonBoyutu = 3;
    int ElemanSayisi = SablonBoyutu * SablonBoyutu;

    int x, y;

    Color Renk;
    int P1, P2, P3, P4, P5, P6, P7, P8, P9;

    for (x = (SablonBoyutu - 1) / 2; x < ResimGenisligi - (SablonBoyutu - 1) / 2; x++) //Resmi
taramaya şablonun yarısı kadar dış kenarlardan içeride başlayacak ve bitirecek.
    {
        for (y = (SablonBoyutu - 1) / 2; y < ResimYuksekligi - (SablonBoyutu - 1) / 2; y++)
        {

            Renk = GirisResmi.GetPixel(x - 1, y - 1);
            P1 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x, y - 1);
            P2 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x + 1, y - 1);
            P3 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x - 1, y);
            P4 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x, y);
            P5 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x + 1, y);
            P6 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x - 1, y + 1);
            P7 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x, y + 1);
            P8 = (Renk.R + Renk.G + Renk.B) / 3;

            Renk = GirisResmi.GetPixel(x + 1, y + 1);
            P9 = (Renk.R + Renk.G + Renk.B) / 3;

            //Hesaplamayı yapan Sobel Temsili matrisi ve formülü.
            int Gx = Math.Abs(-P1 + P3 - 2 * P4 + 2 * P6 - P7 + P9); //Dikey çizgiler
            int Gy = Math.Abs(P1 + 2 * P2 + P3 - P7 - 2 * P8 - P9);  //Yatay Çizgiler

            //if (Gx > 100)
            //    Gx = 255;
            //else
            //    Gx = 0;

            //if (Gy > 100)
            //    Gy = 255;
            //else
            //    Gy = 0;

            int Gxy = Gx + Gy;

            //if (Gxy > Esikleme)
            //    Gxy = 255;
            //else
            //    Gxy = 0;

            //Renkler sınırların dışına çıktıysa, sınır değer alınacak. Negatif olamaz,
formüllerde mutlak değer vardır.
            if (Gx > 255) Gx = 255;
            if (Gy > 255) Gy = 255;
```

```
            if (Gxy > 255) Gxy = 255;

            //int TetaRadyan = 0;
            //if (Gy != 0)
            //    TetaRadyan = Convert.ToInt32(Math.Atan(Gx / Gy));
            //else
            //    TetaRadyan = Convert.ToInt32(Math.Atan(Gx));


            //int TetaDerece = Convert.ToInt32((TetaRadyan * 360) / (2 * Math.PI));

            //if (TetaDerece >= 0 && TetaDerece < 45)
            //    CikisResmiXY.SetPixel(x, y, Color.FromArgb(0, 0, 0));

            //if (TetaDerece >= 45 && TetaDerece < 90)
            //    CikisResmiXY.SetPixel(x, y, Color.FromArgb(0, 255, 0));

            //if (TetaDerece >= 90 && TetaDerece < 135)
            //    CikisResmiXY.SetPixel(x, y, Color.FromArgb(0, 0, 255));

            //if (TetaDerece >= 135 && TetaDerece < 180)
            //    CikisResmiXY.SetPixel(x, y, Color.FromArgb(255, 255, 0));


            CikisResmiX.SetPixel(x, y, Color.FromArgb(Gx, Gx, Gx));
            CikisResmiY.SetPixel(x, y, Color.FromArgb(Gy, Gy, Gy));

            CikisResmiXY.SetPixel(x, y, Color.FromArgb(Gxy, Gxy, Gxy));

        }
    }
    pictureBox2.Image = CikisResmiXY;
    pictureBox3.Image = CikisResmiX;
    pictureBox4.Image = CikisResmiY;
}
```

2. Kodlama

```
private void SOBEL_KENAR2_Click(object sender, EventArgs e)
{
    Color OkunanRenk;
    Bitmap GirisResmi, CikisResmiX, CikisResmiY, CikisResmiXY;
    GirisResmi = new Bitmap(pictureBox1.Image);

    int ResimGenisligi = GirisResmi.Width;
    int ResimYuksekligi = GirisResmi.Height;

    CikisResmiX = new Bitmap(ResimGenisligi, ResimYuksekligi);
    CikisResmiY = new Bitmap(ResimGenisligi, ResimYuksekligi);
    CikisResmiXY = new Bitmap(ResimGenisligi, ResimYuksekligi);

    int SablonBoyutu = 3;
    int ElemanSayisi = SablonBoyutu * SablonBoyutu;


    int x, y, i, j;
    int Gri = 0;

    int[] MatrisX = { -1, 0, 1, -2, 0, 2, -1, 0, 1 };

    int[] MatrisY = { 1, 2, 1, 0, 0, 0, -1, -2, -1 };

    int RenkX, RenkY, RenkXY;
```

```
    for (x = (SablonBoyutu - 1) / 2; x < ResimGenisligi - (SablonBoyutu - 1) / 2; x++) //Resmi
taramaya şablonun yarısı kadar dış kenarlardan içeride başlayacak ve bitirecek.
    {
        for (y = (SablonBoyutu - 1) / 2; y < ResimYuksekligi - (SablonBoyutu - 1) / 2; y++)
        {
            int toplamGriX = 0, toplamGriY = 0;

            //Şablon bölgesi (çekirdek matris) içindeki pikselleri tarıyor.
            int k = 0; //matris içindeki elemanları sırayla okurken kullanılacak.

            for (i = -((SablonBoyutu - 1) / 2); i <= (SablonBoyutu - 1) / 2; i++)
            {
                for (j = -((SablonBoyutu - 1) / 2); j <= (SablonBoyutu - 1) / 2; j++)
                {
                    OkunanRenk = GirisResmi.GetPixel(x + i, y + j);

                    Gri = (OkunanRenk.R + OkunanRenk.G + OkunanRenk.B) / 3;

                    toplamGriX = toplamGriX + Gri * MatrisX[k];

                    toplamGriY = toplamGriY + Gri * MatrisY[k];

                    k++;
                }

            }

            RenkX = Math.Abs(toplamGriX);
            RenkY = Math.Abs(toplamGriY);
            RenkXY = Math.Abs(toplamGriX) + Math.Abs(toplamGriY);

            //============================================================
            //Renkler sınırların dışına çıktıysa, sınır değer alınacak.
            if (RenkX > 255) RenkX = 255;
            if (RenkY > 255) RenkY = 255;
            if (RenkXY > 255) RenkXY = 255;


            if (RenkX < 0) RenkX = 0;
            if (RenkY < 0) RenkY = 0;
            if (RenkXY < 0) RenkXY = 0;

            //============================================================

            CikisResmiX.SetPixel(x, y, Color.FromArgb(RenkX, RenkX, RenkX));
            CikisResmiY.SetPixel(x, y, Color.FromArgb(RenkY, RenkY, RenkY));
            CikisResmiXY.SetPixel(x, y, Color.FromArgb(RenkXY, RenkXY, RenkXY));
        }
    }
    pictureBox2.Image = CikisResmiX;
    pictureBox3.Image = CikisResmiY;
    pictureBox4.Image = CikisResmiXY;
}
```